

Министерство цифрового развития

Федеральное государственное общеобразовательное учреждение высшего
образования

«Сибирский Государственный Университет Телекоммуникаций и
Информатики»
(СибГУТИ)

Кафедра прикладной математики и кибернетики

Вариант 2.

Отчет по практической работе №3.

«Модульное тестирование программ на языке Python в среде VS Code»

Работу выполнил студент группы ИП213:

Жуйков С. А.

Соколовский Н. А.

Терновский Д. Р.

Работу проверил ст. преподаватель кафедры ПМиК:

Агалаков А. А.

г. Новосибирск, 2025 г.

Цель работы

Сформировать практические навыки разработки тестов и модульного тестирования на языке Python с помощью средств автоматизации Visual Studio Code.

Задание

Разработайте на языке Python класс, содержащий набор функций в соответствии с вариантом задания.

Разработайте тестовые наборы данных по критерию С2 для тестирования функций класса.

Протестировать функции с помощью средств автоматизации модульного тестирования Visual Studio Code.

Провести анализ выполненного теста и, если необходимо отладку кода.

Написать отчёт о результатах проделанной работы.

Вариант, №	Спецификации методов
2.	Метод находит максимальное значение из трёх значений целых переменных.
	Метод получает число a. Формирует и возвращает целое число b, полученное из четных значений разрядов целого числа a, следующих в обратном порядке. Например: a = 12345, b = 42.
	Метод получает целое число n. Находит и возвращает минимальное значение r, среди значений разрядов целого числа a. Например: n = 62543, r = 2
	Метод получает двумерный массив целых переменных A. Отыскивает и возвращает сумму нечётных значений компонентов массива, лежащих ниже главной диагонали.

Управляющий граф программы

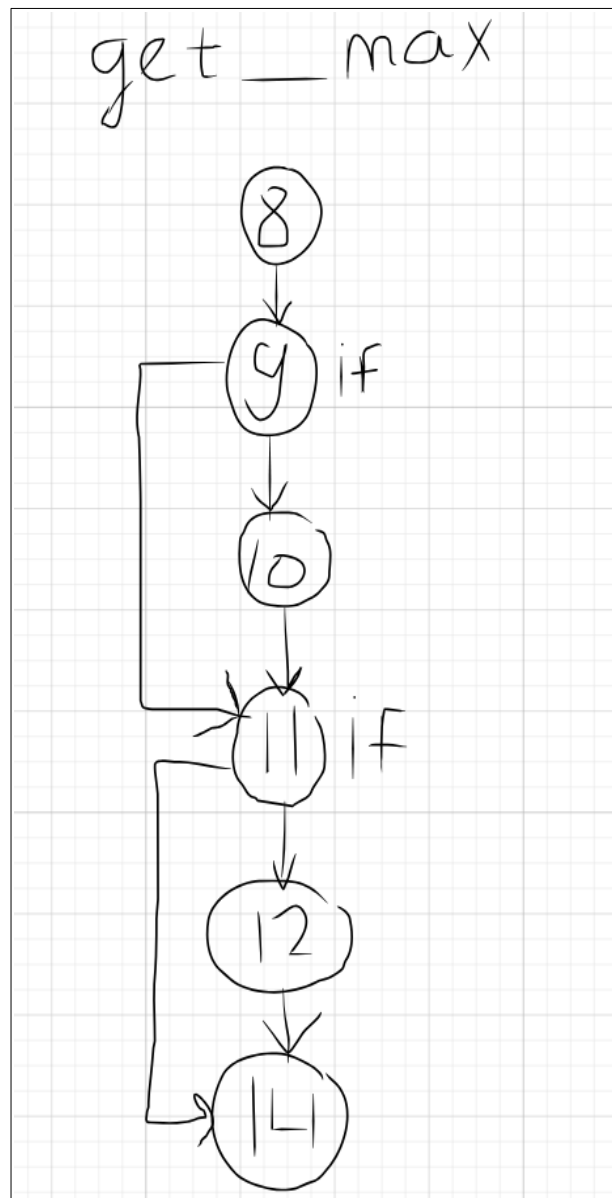


Рис. 1 — Управляющий граф для метода get_max.

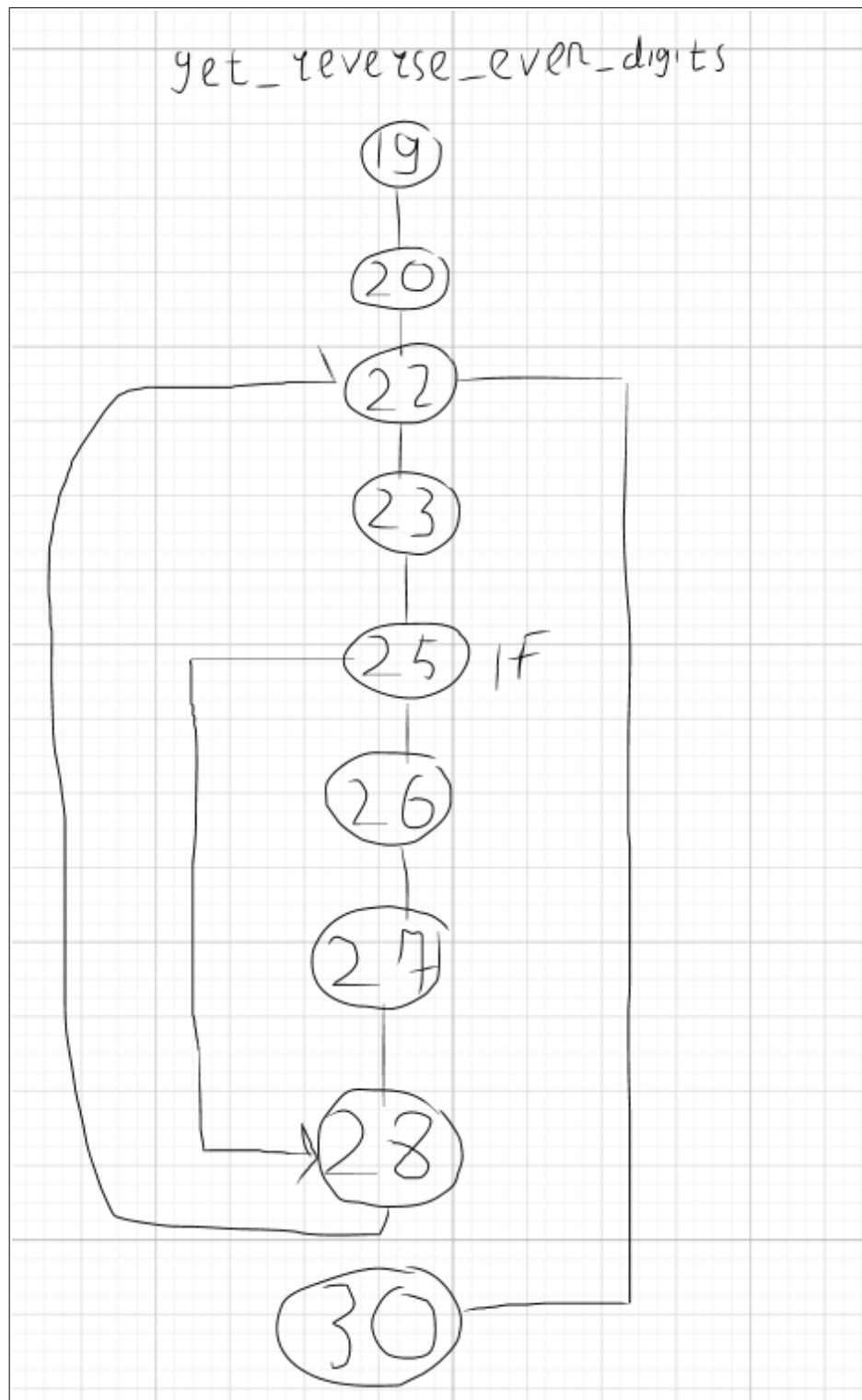


Рис. 2 — Управляющий граф для метода `get_reverse_even_digits`.

get_min_digit

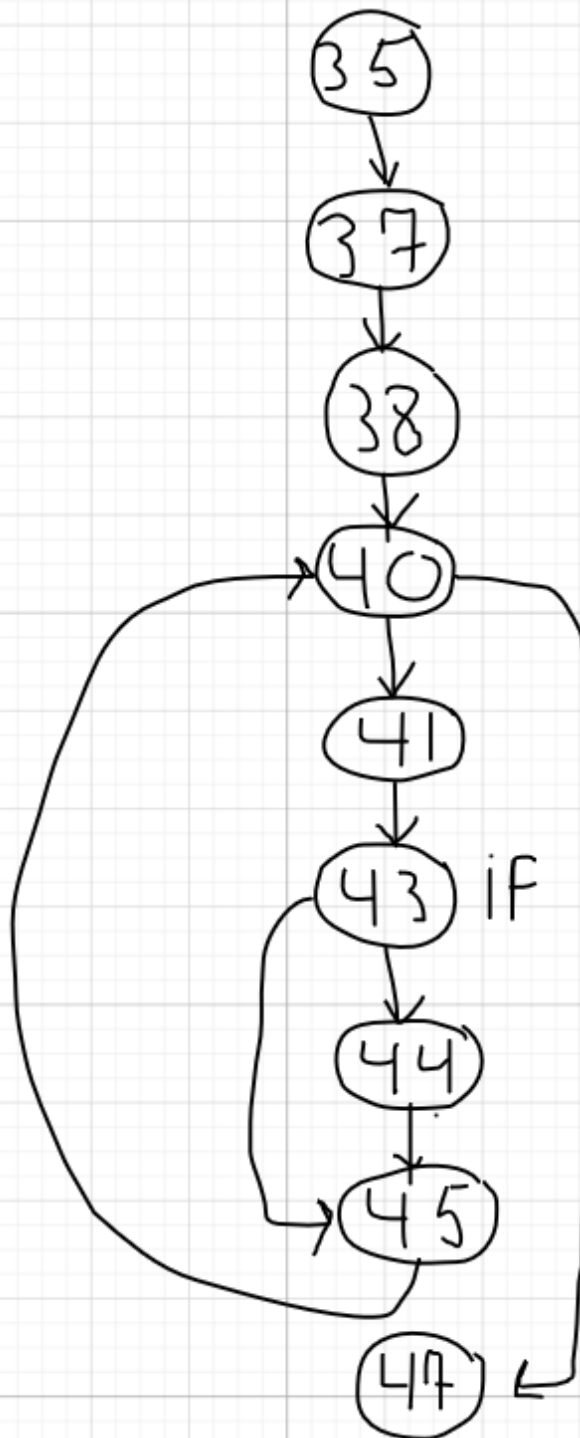


Рис. 3 — Управляющий граф для метода `get_min_digit`.

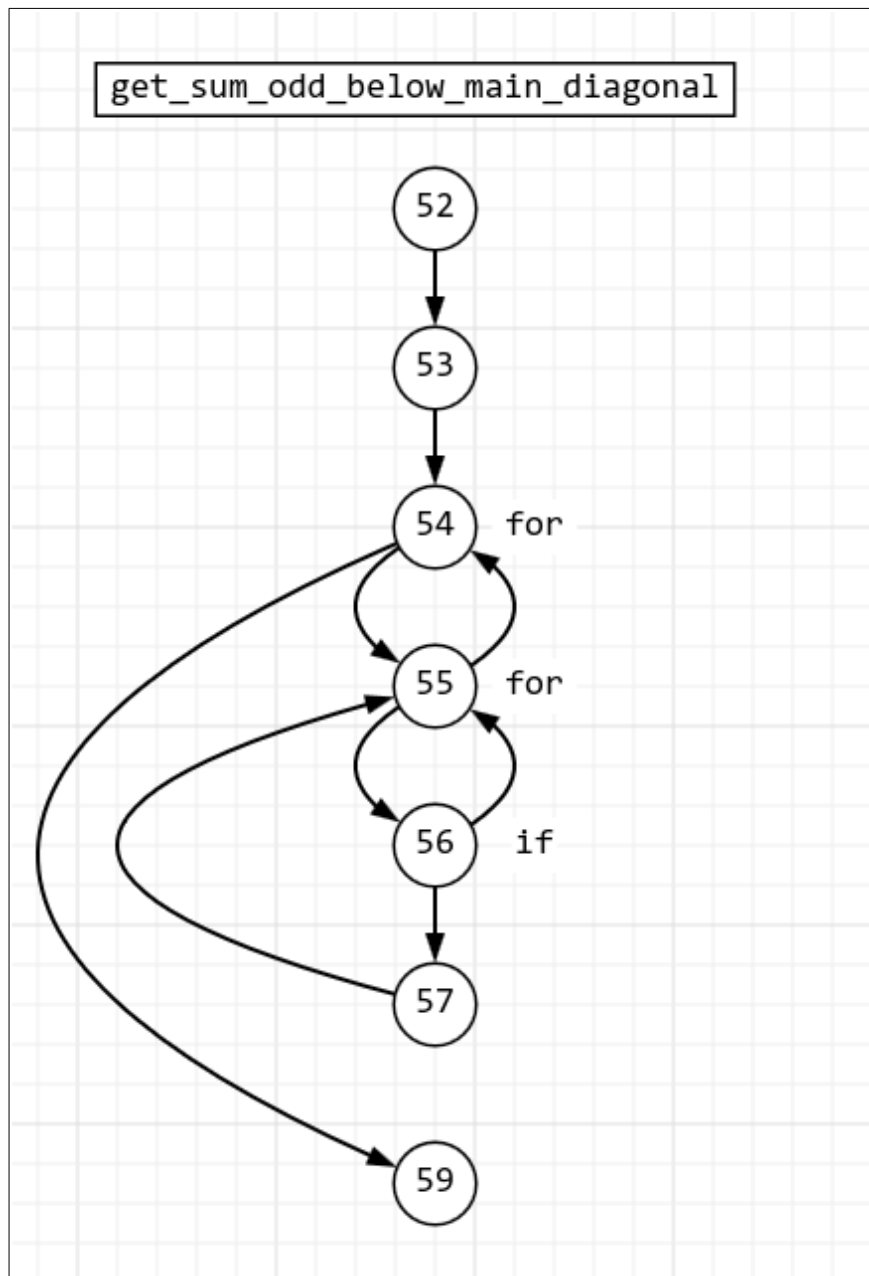


Рис. 4 — Управляющий граф для метода `get_sum_odd_below_main_diagonal`

Листинг программы

```
1  from typing import List
2
3  class Operations:
4
5      @staticmethod
6      def get_max(num1: int, num2: int, num3: int) -> int:
7
8          max: int = num1
9          if (num2 >= max):
10             max = num2
11          if (num3 >= max):
12             max = num3
13
14          return max
15
16      @staticmethod
17      def get_reverse_even_digits(number: int) -> int :
18
19          result: int = 0
20          number = abs(number)
21
22          while (number > 0):
23              digit: int = number % 10
24
25              if (digit % 2 == 0):
26                  result *= 10
27                  result += digit
28                  number //= 10
29
30          return result
31
32      @staticmethod
33      def get_min_digit(number: int) -> int:
34
35          number = abs(number)
36
37          min_digit: int = number % 10
38          number //= 10
39
40          while (number > 0):
41              digit = number % 10
42
43              if (digit < min_digit):
44                  min_digit = digit
45                  number //= 10
46
47          return min_digit
48
49      @staticmethod
50      def get_sum_odd_below_main_diagonal(matrix: List[List[int]]) -> int:
51
52          sum_val = 0
53          size = len(matrix)
54          for i in range(size):
55              for j in range(i):
56                  if matrix[i][j] % 2 != 0:
57                      sum_val += matrix[i][j]
58
59          return sum_val
```



```
1 import random
2 from typing import List
3
4 from src.operations import Operations
5
6 if __name__ == "__main__":
7
8     a = random.randint(0, 1000)
9     b = random.randint(0, 1000)
10    c = random.randint(0, 1000)
11    print(f"max({a}, {b}, {c}) = {Operations.get_max(a, b, c)}")
12    print()
13
14    number= random.randint(0, 100000)
15    print(f"number = {number}, reversed_even_digits =
{Operations.get_reverse_even_digits(number)}")
16    print()
17
18    print(f"number = {number}, min_digit = {Operations.get_min_digit(number)}")
19    print()
20
21    rows = 3
22    cols = 3
23    matrix = [[random.randint(0, 1000) for _ in range(cols)] for _ in range(rows)]
24
25    print(f"matrix = {matrix}")
26    print(f"sum of odd elems below main diagonal =
{Operations.get_sum_odd_below_main_diagonal(matrix)}")
```

```

1 import unittest
2 from typing import List
3
4 from src.operations import Operations
5
6 class TestOperations(unittest.TestCase):
7
8     """ Тесты для метода Operations.get_max """
9     def test_max_first_param_is_greater(self):
10         self.assertEqual(Operations.get_max(3, 2, 1), 3)
11
12     """ Тесты для метода Operations.get_reverse_even_digits """
13     def test_get_reverse_even_digits_with_negative_number(self):
14         self.assertEqual(Operations.get_reverse_even_digits(-1256), 62)
15
16     def test_get_reverse_even_digits_having_even_digits(self):
17         self.assertEqual(Operations.get_reverse_even_digits(124356), 642)
18
19     def test_get_reverse_even_digits_not_having_even_digits(self):
20         self.assertEqual(Operations.get_reverse_even_digits(1357), 0)
21
22     def test_get_reverse_even_digits_with_zero(self):
23         self.assertEqual(Operations.get_reverse_even_digits(0), 0)
24
25     """ Тесты для метода Operations.get_min_digit """
26     def test_get_min_digit_with_negative_number(self):
27         self.assertEqual(Operations.get_min_digit(-34165), 1)
28
29     def test_get_min_digit_with_positive_number(self):
30         self.assertEqual(Operations.get_min_digit(34265), 2)
31
32     def test_get_min_digit_single_digit(self):
33         self.assertEqual(Operations.get_min_digit(7), 7)
34
35     def test_get_min_digit_with_zero(self):
36         self.assertEqual(Operations.get_min_digit(0), 0)
37
38
39     """ Тесты для метода Operations.get_sum_below_main_diagonal """
40     def test_get_sum_odd_below_main_diagonal_with_correct_param(self):
41         matrix = [[1, 2, 4], [3, 5, 6], [7, 8, 9]]
42         self.assertEqual(Operations.get_sum_odd_below_main_diagonal(matrix), 10)
43
44     def test_get_sum_odd_with_no_odd_numbers(self):
45         matrix = [[2, 4, 6], [8, 10, 12], [14, 16, 18]]
46         self.assertEqual(Operations.get_sum_odd_below_main_diagonal(matrix), 0)
47
48     def test_get_sum_odd_with_1x1_matrix(self):
49         self.assertEqual(Operations.get_sum_odd_below_main_diagonal([[5]]), 0)
50
51     def test_get_sum_odd_below_main_diagonal_with_multiple_odd_numbers(self):
52         matrix = [[1, 2, 3], [5, 6, 7], [9, 10, 11]]
53         self.assertEqual(Operations.get_sum_odd_below_main_diagonal(matrix), 14)
54
55 if __name__ == "__main__":
56     unittest.main()

```

Результаты выполнения модульных тестов

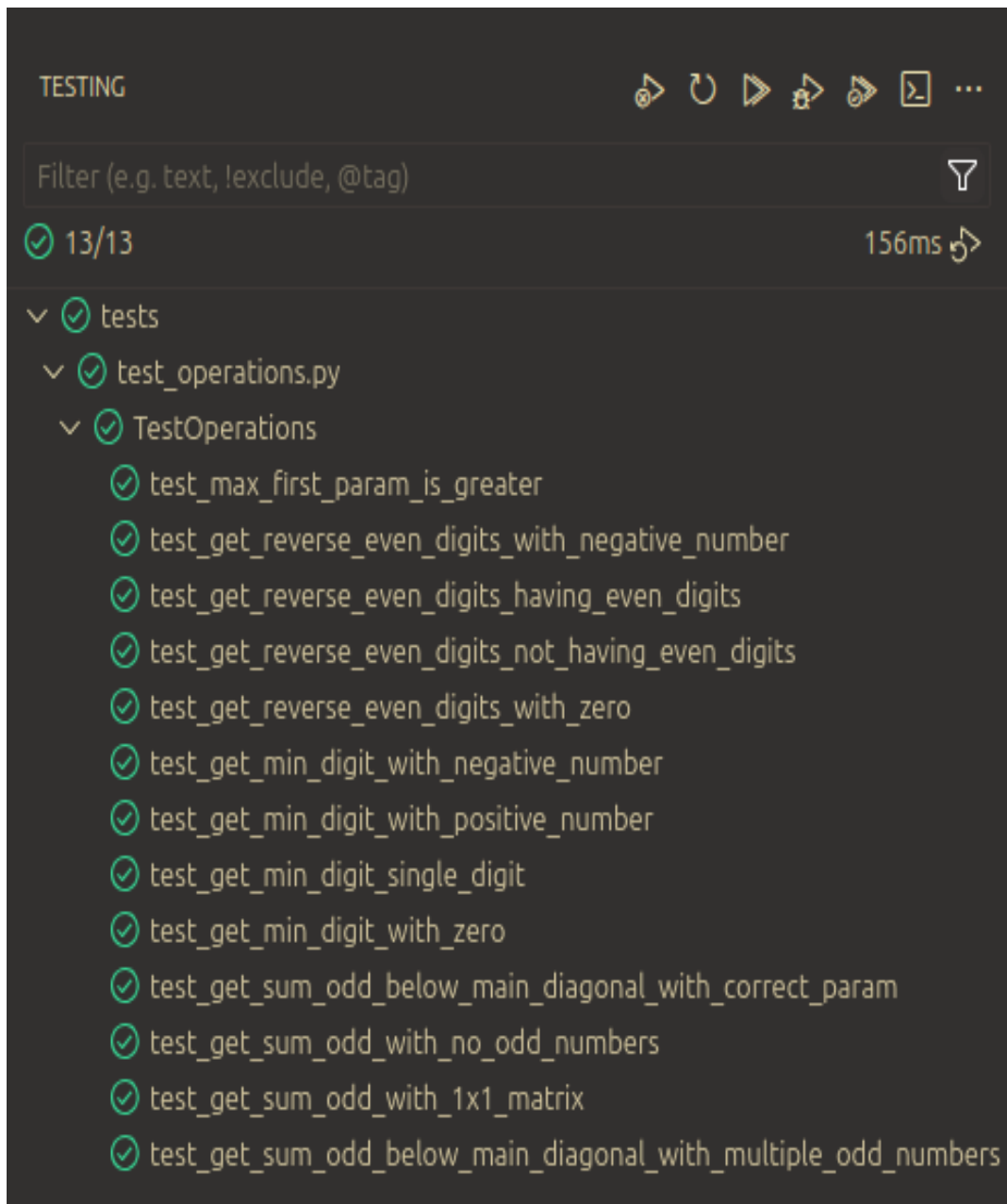


Рис. 5 — Результаты модульных тестов для класса Operations.

Результаты покрытия разработанного кода тестами

```
~/sibsutis/modern_programming_technologies/lab3
• > coverage run -m unittest discover -s tests
.....
-----
Ran 13 tests in 0.001s

OK

~/sibsutis/modern_programming_technologies/lab3
• > python -m coverage report
Name                               Stmts   Miss  Cover
-----
src/operations.py                   39      0   100%
-----
TOTAL                               39      0   100%

~/sibsutis/modern_programming_technologies/lab3
○ > █
```

Рис. 6 — Результат покрытия кода тестами с помощью модуля coverage.

Coverage report: 100%

Files Functions Classes

coverage.py v7.10.7, created at 2025-10-16 01:09 +0700

File ▲	function	statements	missing	excluded	coverage
src/operations.py	Operations.get_max	4	0	0	100%
src/operations.py	Operations.get_reverse_even_digits	9	0	0	100%
src/operations.py	Operations.get_min_digit	9	0	0	100%
src/operations.py	Operations.get_sum_odd_below_main_diagonal	7	0	0	100%
src/operations.py	(no function)	10	0	0	100%
Total		39	0	0	100%

coverage.py v7.10.7, created at 2025-10-16 01:09 +0700

Рис. 7 — Результаты покрытия кода тестами в виде html-страницы.

Выводы по проделанной работе

В результате выполнения практической работы разработан класс, методы которого соответствуют предоставленной к практической работе спецификации. Разработан набор модульных тестов для проверки корректности работы методов разработанного класса. Получены начальные навыки разработки модульных тестов для тестирования методов классов и выполнения модульного тестирования на языке Python с помощью средств автоматизации Visual Studio Code.